认知重建：Speckit 用了三个月，我放弃了——走出工具很强但用不好的困境

原创 腾讯程序员 腾讯技术工程 2026年1月9日 01:36 广东 1148人



作者：rickyshou

2025 年 AI 编程工具遍地开花，但一个尴尬的现实是：工具越来越强，预期越来越高，落地却越来越难——speckit 的规范流程在企业需求的"千层套路"、海量代码面前显得理想化，上下文窗口频繁爆满让复杂任务半途而废，每次做类似需求还是要花同样的时间因为知识全在人脑里。本文记录了我从踩坑规范驱动工具，到借鉴 Anthropic 多 Agent 协作架构、融合上下文工程与复合工程理念，最终实现边际成本递减、知识持续复利的完整历程。如果你也在"AI 工具明明很强但就是用不好"的困境中挣扎，或许能找到一些共鸣。附带还有新的工作流下人的工作模式转变思考～

一、起点：规范驱动开发的美好承诺

1.0 团队的 AI Coding 起点

先交代一下背景：我所在的是一个**后端研发团队**，日常工作以存量项目迭代为主，涉及多个微服务的协作开发。

2024 年中，团队开始尝试 AI 辅助编程。最初的体验是：

短上下文场景效果不错：

- 写一个独立函数、实现一个工具方法——AI 表现良好
- 简单的代码补全、格式化、注释生成——确实提效

但规模化复用始终没起来：

- 当时只有三种触发类型的 rules（早期 rules 时代）
- 虽然提出过"在基础 agent 之上封装 agent"的想法
- 但几个月过去，仍然没有太多人真正动起来

原因分析：

- 规范没有形成共识——每个人对"怎么用好 AI"理解不同

- 对 AI 工程化没有标准认识——不知道该往哪个方向努力
- 提示词复用习惯没建立——好的 prompt 停留在个人经验，没有沉淀

这个困境促使我开始探索外部方案：有没有已经成熟的"AI 编程工程化"方法论？有没有可以直接借鉴的最佳实践？

带着这些问题，我遇到了 speckit 和 openspec。

1.1 遇见 speckit：AI 编程的"正确打开方式"？

2024 年开始，AI 编程助手如雨后春笋般涌现。Copilot、Cursor、Claude 让很多人第一次体验到了"AI 写代码"的魔力。但兴奋之后，问题也随之而来：

- AI 生成的代码质量参差不齐
- 需求理解经常偏离预期
- 缺乏持续性，上下文丢失严重
- 改一处坏十处，维护成本高

正当我被这些问题困扰时，遇到了 **speckit**——一个规范驱动开发（Spec-Driven Development, SDD）工具包。

speckit 的理念很吸引人：

```
规范即代码 → 规范直接生成实现，而非仅作为指导文档
权力倒置 → 代码服务于规范，而非规范服务于代码
测试优先 → 强制 TDD，不可协商地要求先写测试
```

它定义了一套清晰的 5 阶段流程：

```
Constitution → Specify → Plan → Tasks → Implement
   （宪章）      （规范）   （计划）（任务）  （实施）
```

每个阶段对应一个命令，依次执行：创建项目宪章和开发原则 → 定义需求和用户故事 → 创建技术实现计划 → 生成可执行的任务列表 → 执行所有任务构建功能。

再加上 9 条不可变的架构原则（库优先、CLI 接口、测试优先、简洁性、反抽象...），7 层 LLM 输出约束机制，防止过早实现、强制标记不确定性、结构化自检...

这不就是 AI 编程的"工程化正确答案"吗？

带着这样的期待，我开始在项目中尝试落地。

1.2 openspec：另一种优雅的尝试

除了 speckit，我还研究了 **openspec**——一个更轻量的规范驱动框架：

```
Specs as Source of Truth → specs/ 目录始终反映系统当前真实状态
Changes as Proposals → 所有修改先以提案形式存在，经确认后实施
Lock Intent → AI 编码前通过明确规范锁定意图
```

openspec 的 Delta 机制设计得很巧妙：不同于直接存储完整的"未来状态"，它只存储**变更操作**本身（ADDED/MODIFIED/REMOVED/RENAMED）。归档时通过语义名称匹配来定位需求，避免了 Git Merge 常见的位置冲突问题。同时采用 Fail-Fast 机制，在写入前做完整冲突检测，保证不会产生半完成状态。

两个工具，两种风格，但都指向同一个目标：让 AI 编程更可控、更规范。



腾讯技术工程

腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。

688篇原创内容

公众号

二、碰壁：理想流程遭遇企业现实

2.1 一个真实需求的"千层套路"

让我用一个真实的 12 月活动需求来说明问题：

协作复杂度：

- 跨 BG、跨前后端、跨 FT、跨项目、跨小组、跨服务
- 跨部门合作接口因合规要求变来变去，迟迟给不到位
- 雅典娜平台上接近 20 种商品类型，全得人工一个个配
- 活动流程必须按"玩法引擎"的方法论来拆解
- 技术方案得按习惯写在 iWiki 里

并行任务流：

```
同时处理：
├── 找产品确认商品细节
├── 找运营确认玩法逻辑
├── 找跨团队研发对齐接口
├── 找跨项目研发对齐交互
└── 内部技术方案评审
```

方案设计的"考古"需求：

- 某个商品创建、资产查看以前有什么坑？

- 现在的玩法能力有哪些？能不能直接用？
- 导航小结页到底是啥？怎么让它弹 Banner？

写代码前的"九九八十一难"：

```
前置任务链：
├── 玩法引擎：依赖数据、激励动作要在引擎仓库里实现
├── 外部依赖：关联的代码改动在其他服务里
├── 配置中心：要去阿波罗（Apollo）配配置
├── 雅典娜：商品场景得先配好（早期没数据还得 Mock）
└── 数据库：涉及表变更，得去测试环境操作
```

执行中的细节坑：

- 阿波罗配置有个坑，该怎么绕过去？
- 规则引擎的语法到底怎么写？
- 商品发放操作是重点，具体发到哪个钱包？

2.2 speckit 流程 vs 企业现实

把 speckit 的理想流程放到这个场景里：

```
speckit 假设的流程：
Constitution → Specify → Plan → Tasks → Implement
     ↓            ↓        ↓       ↓         ↓
  一次性定义   一次性写清   线性规划   任务分解   按序实施

企业现实：
多方博弈 → 动态调整 → 并行推进 → 持续扯皮 → 边做边改
    ↓          ↓          ↓          ↓          ↓
 需求会变   方案会改   依赖会卡   资源会抢   意外会来
```

核心矛盾：speckit 假设需求是清晰的、可一次性规划的，但企业真实需求是动态的、多方博弈的、持续变化的。

2.3 openspec 的 Delta 机制也救不了

openspec 的"提案→审查→归档"流程看起来更灵活，但：

- **假设需求可以"提案化"**：实际上外部接口因合规变来变去，5 个维度同时推进相互依赖，评审中发现问题需要立即改方案
- **人工介入成本高**：Delta 与主 Spec 冲突时报错终止，复杂冲突需要人工解决，而人的认知窗口有限。具体来说，`openspec archive` 会在以下情况直接报错退出：

- MODIFIED 引用的需求在主 Spec 中不存在（可能被别人删了或改名了）
- ADDED 的需求在主 Spec 中已存在（别的分支先合入了同名需求）
- RENAMED 的源名称不存在，或目标名称已被占用
- 同一个需求同时出现在 MODIFIED 和 REMOVED 中（逻辑矛盾）

  这些冲突没有自动解决策略，CLI 只会打印类似 `MODIFIED failed for header "### Requirement: xxx" - not found` 的错误信息，然后终止。你需要：手动打开两个文件对比、理解冲突原因、决定保留哪个版本、手工修改 Delta 文件、重新执行归档。整个过程要求你同时在脑中持有"主 Spec 当前状态"和"Delta 期望变更"两套信息——这对认知负担是很大的挑战

- **强依赖命名的脆弱性**：产品叫"用户激励"，运营叫"活动奖励"，研发叫"商品发放"——同一个需求在不同阶段有不同表述

2.4 最致命的问题：无法应对"考古"需求

speckit 和 openspec 都有一个共同盲区：**流程从零开始**。

```
speckit 流程:
Constitution 定义原则 → Specify 定义需求 → Plan 设计方案 → ...

但真实需求必须"考古":
├── 这个商品创建以前有什么坑?
├── 现有玩法能力有哪些?
├── 导航小结页的 Banner 怎么弹?
├── Apollo 配置有什么特殊处理?
└── 雅典娜 20 种商品类型的配置方式各不同
```

缺失能力：没有"上下文检索"机制，无法自动关联历史经验、已有能力、已知陷阱。

AI 生成 spec 时能看到的：

- ✅ 代码仓库
- ✅ project.md/Constitution
- ✅ 用户意图

AI 看不到（但需要知道）的：

- ❌ 业务边界（涉及哪些服务？）
- ❌ 历史经验（以前怎么做的？有什么坑？）
- ❌ 配置规范（Apollo 特殊要求？）
- ❌ 平台知识（雅典娜 20 种商品配置注意事项）

- ❌ 协作约束（依赖其他团队接口？合规要求？）

结果：依赖人 review 时逐步想起来告诉 AI，45 分钟 + 持续的认知负担。

2.5 AI 工程化如何破局？（预告）

面对上述问题，AI 工程化的解决思路是什么？这里先做个预告，详细方案见第五节。

企业现实问题	speckit/openspec 的困境	AI 工程化的解法
需求动态变化	假设一次性规划，变更成本高	需求以"进行中"状态管理，支持随时调整，阶段性沉淀
多线并行博弈	线性流程，Delta 冲突报错终止	Agent 自主决策路由，Skill 独立执行，不强依赖顺序
考古需求	无上下文检索，AI 只能看到代码	`context/` 分层管理历史经验，按阶段自动加载
配置/平台知识	需要人 review 时口述	沉淀为 `context/tech/`，AI 执行时主动提醒
冲突解决成本	人工对比、手工修改、认知负担重	不依赖"合并"，而是"覆盖+沉淀"，冲突时 AI 辅助决策
边际成本恒定	每次 45 分钟，无复利	首次建立 context，后续复用，边际成本递减

核心差异：

```
speckit/openspec 的思路：
规范化流程 → 约束 AI 行为 → 期望产出质量
    ↓
问题：流程本身不适配企业现实，约束越多越僵化

AI 工程化的思路：
上下文完整性 → AI 决策质量 → 自动沉淀经验 → 下次更好
    ↓
解法：不是约束 AI，而是给 AI 完整信息 + 让知识复利
```

一个具体例子——同样是"商品发放"需求：

```
speckit 模式（第 3 次做）：
1. Constitution → 写项目原则（已有，跳过）
2. Specify → 写需求规范（45 分钟，人逐步想起遗漏告诉 AI）
```

```
3. Plan → 写技术方案（人提醒：Apollo 有坑、钱包要区分）
4. Tasks → 生成任务（人补充：雅典娜配置注意事项）
5. Implement → 执行（遇到问题再排查）
耗时：45 分钟 + 排查时间，知识留在人脑

AI 工程化模式（第 3 次做）：
1. /req-dev "商品发放需求"
2. Agent 识别意图 → 自动加载 context/experience/商品发放历史问题.md
3. Agent 提醒："历史上有钱包选择、Apollo 配置、雅典娜商品类型三个坑点"
4. 人确认："对，继续"
5. Skill 执行 → 自动校验 → 生成代码 → 沉淀新发现
耗时：10 分钟，知识沉淀到 context/
```

后续章节将详细展开这套方案的设计原理和落地实践。

三、反思：从第一性原理重新审视

3.1 人的认知局限是刚性约束

实话实说，我的脑容量有限：

- **记性不好**：只能记住关键的大方向，具体细节过脑就忘
- **专注窗口小**：同时关注的信息有限，必须采用"专注单任务+全局索引"策略

我的日常工作模式（经过各种场景检验的最优路径）：

- **任务管理（外挂大脑）**：Todo List 分优先级（红色紧急/黄色进行中/绿色完成/无色未开始）
- **备忘录**：记录死记硬背的内容（打包命令、数据库 IP 密码、文档散落信息）
- **桌面即上下文**：N 个桌面窗口，每个窗口对应一个垂直领域
- **复杂任务 SOP 化**：脑内计划 + 执行机器模式 + 文档跟踪
- **简单任务 Fire and Forget**：低频低思考成本事项秒回即忘

这套土办法是经过检验的最优路径。如果硬套 speckit/openspec 的范式，反而会丢掉这些 SOP，得不偿失。

3.2 执行过程的知识价值被忽视

speckit 和 openspec 都只关注"规范"（Spec）和"结果"（Code），忽视"过程"（Process）。

但真实价值恰恰在过程中：

```
执行 → 有问题 → 验证 → 排查 → 继续执行
                    ↓
```

```
排查信息往往没被记录
        ↓
时间一久或换人，下次重新排查
```

这个循环中的排查信息，才是最宝贵的知识！

3.3 边际成本恒定是致命缺陷

```
Speckit 模式：
第 1 次商品发放需求：45 分钟（人逐步想起遗漏）
第 2 次商品发放需求：45 分钟（人 AGAIN 逐步想起遗漏）
第 n 次商品发放需求：45 分钟（还是要想，还是那么久）

边际成本恒定，无复利效应。
知识在哪里？在人脑里，每次都要重新想起来。
```

这与我期望的"越用越快"完全相反。

四、转折：遇见复合工程与上下文工程

4.1 复合式工程：让每一步都成为下一步的基石

在探索过程中，我接触到了"复合式工程"（Compounding Engineering）的理念。这个概念来自 Claude Code 团队与 Every 团队的实践交流，并在 Every 团队开源的 **Compound Engineering Plugin** 中得到了系统化实现——这是一个包含 27 个 Agent、19 个 Command、13 个 Skill 的完整 AI 辅助开发工具包。

定义"复合式工程"

"复合式工程"的核心目标非常明确：让每一单元的工程工作使后续工作变得更容易，而非更难。

```
传统开发：累积技术债务 → 每个功能增加复杂性 → 代码库越来越难维护
复合工程：每个功能产出文档模式 → 创建可复用组件 → 建立减少决策疲劳的约定 → 知识在团队中复合增长
```

与传统工程中每增加一个功能都会增加系统复杂度和维护成本不同，"复合式工程"追求的是一种"复利"效应，让系统的能力随着时间推移指数级增长。

核心工作流循环：Plan → Work → Review → Compound

Compound Engineering Plugin 设计了一个闭环的工作流循环：

```
```



- **Plan**：多代理并行研究仓库模式、最佳实践、框架文档，输出结构化计划
- **Work**：系统性执行计划，边做边测，质量内建
- **Review**：多代理并行审查（安全、性能、架构等），输出分级 Todo
- **Compound**：这是复合工程的核心——将解决的问题结构化记录，形成团队知识资产

> 完整实现参见：**Compound Engineering Plugin**

为什么叫"Compound"?

```
第一次解决 "N+1 query in brief generation" → Research (30 min)
文档化 → docs/solutions/performance-issues/n-plus-one-briefs.md (5 min)
下次类似问题 → Quick lookup (2 min)
知识复合 → Team gets smarter

Each unit of engineering work should make subsequent units of work easier—not harder.
```

实现机制：知识复合的典型场景

实现复合工程的关键，在于建立系统化的知识沉淀机制。以下是几个典型场景：

场景 1：Agent 重复犯同类错误

```
触发：发现 Agent 在某类问题上反复出错
沉淀：将教训写入 AGENTS.md / CLAUDE.md / 系统提示词
效果：该类错误不再发生，无需人工提醒
```

场景 2：某类问题需要频繁人工检查

```
触发：Code Review 时反复指出同类问题
沉淀：创建 Lint 规则 / Pre-commit Hook / CI 检查
效果：问题在提交前自动拦截，减少人工负担
```

场景 3：复杂流程被多次执行

```
```

```
触发：某个多步骤操作被团队重复执行
沉淀：封装为 Skill / Command / Agent
效果：一键触发标准化流程，新人也能执行专家级操作
```

场景 4：解决了一个有价值的问题

```
触发：花了较长时间解决某个棘手问题
沉淀：结构化记录到 context/experience/ 目录
效果：下次遇到类似问题，Agent 自动加载相关经验
```

这些场景的共同特点是：**在问题解决的当下立即沉淀**，而不是事后补文档。

Claude 团队的复合工程应用案例

以下是 Every 团队和 Anthropic 内部使用复合工程的真实案例：

案例 1："@claude，把这个加到 claude.md 里"

当有人在 PR 里犯错，团队会说："@claude，把这个加到 claude.md 里，下次就不会再犯了。"或者："@claude，给这个写个测试，确保不会回归。"通过这种方式，错误转化为系统的免疫能力。

案例 2：100% AI 生成的测试和 Lint 规则

Claude Code 内部几乎 100% 的测试都是 Claude 写的。坏的测试不会被提交，好的测试留下来。Lint 规则也是 100% Claude 写的，每次有新规则需要，直接在 PR 里说一句："@claude，写个 lint 规则。"

案例 3：十年未写代码的经理

经理 Fiona 十年没写代码了，加入团队第一天就开始提交 PR。不是因为她重新学会了编程，而是因为 Claude Code 里积累了所有团队的实践经验——系统"记得"怎么写代码。

案例 4：内置记忆系统

把每次实现功能的过程——计划怎么制定的、哪些部分需要修改、测试时发现了什么问题、哪些地方容易遗漏——全部记录下来，编码回所有的 prompts、sub-agents、slash commands。这样下次别人做类似功能时，系统会自动提醒："注意，上次这里有个坑。"

成果：一个自我进化的开发伙伴

这一范式带来的最终效果是惊人的。它将 AI 从一个被动执行命令的工具，转变为一个能够从经验中持续学习、并让整个开发流程效率不断"复利"增长的开发伙伴。

为什么这解决了古老的知识管理问题

传统的知识管理困境：

```
```

```
方式 1：写文档
问题：没人看。写完就过时。维护成本高。

方式 2：靠人传授
问题：老人离职知识断层。新人上手慢。传授效率低。

方式 3：代码注释
问题：注释会过时。只能解释"是什么"，难以解释"为什么这么做"和"以前踩过什么坑"。
```

复合工程的答案：**把知识编码进工具，让工具在正确的时刻主动提醒你。**

```
不是：写一份"商品发放注意事项"文档，期望大家会看
而是：在 context/experience/商品发放历史问题.md 里记录，
      Agent 在执行商品发放需求时自动加载，主动提醒

不是：靠老人口头传授"Apollo 配置有个坑"
而是：把坑编码到 skill 里，执行时自动校验

不是：在代码里写注释"这里要注意 XX"
而是：让 AI 在生成代码前就已经知道要注意 XX
```

关键设计模式

从 Compound Engineering Plugin 中可以提炼出三个核心设计模式：

模式	核心思想	价值
并行代理	多角度分析时启动多个专业代理，合并结果后继续	提高分析覆盖度和效率
意图路由	入口统一，根据意图自动路由到具体工作流	降低用户认知负担
知识复合	问题解决 → 文档化 → 未来查找 → 团队变聪明	边际成本递减

我的实践：基于工具架构的知识复合

基于复合工程理念，我设计了一套 AI 工程工具架构来实现知识的持续沉淀与复用：

工具架构：

```
用户输入 → Command（入口）→ Agent（决策层）→ Skill（执行层）
                                  ↓
                            意图识别、流程路由
                                  ↓
                            调用具体 Skill 执行
```

```
                                    ↓
                        experience-index（经验检索）
```

- **Command**：用户交互入口，如 `/req-dev`、`/optimize-flow`
- **Agent**：自主决策，智能判断意图，可调用多个 Skill
- **Skill**：固化流程，执行具体操作步骤

知识复合的两条路径：

```
路径 1：经验沉淀（/optimize-flow）
用户发现规律 → experience-depositor Agent → 识别规则类型 → 写入规则文件
                                                                ↓
                                                  context-rules.md（上下文映射）
                                                  risk-rules.md（风险识别）
                                                  service-rules.md（服务补全）
                                                  pattern-rules.md（代码模式）

路径 2：经验检索（experience-index Skill）
需求分析/方案设计/代码编写前 → 自动检索匹配规则 → 加载相关 Context、提示风险、建议服务
```

复利效应示例：

```
第 1 次做支付需求：45 分钟（边做边踩坑）
    ↓ 沉淀规则：/optimize-flow "支付需求要加载 payment-service.md 并提示资金安全"

第 2 次做支付需求：15 分钟（experience-index 自动加载背景、提示风险）
    ↓ 沉淀更多规则：错误处理模式、服务依赖关系

第 N 次做支付需求：5 分钟（系统已积累完整的支付领域知识）
```

与传统文档的本质区别：

```
传统文档：写完没人看，看了也找不到对的时机
AI 工程化：experience-index 在正确的时刻自动检索，主动推送给 Agent
```

这就是为什么"知识应该沉淀到工具"不是一句口号，而是有实际 ROI 的工程决策。

对长期任务工程设计的启示

Compound Engineering Plugin 为 AI 工程化提供了极好的参考蓝图：

维度	启示
任务分解	阶段化执行（Plan → Work → Review → Compound），并行化处理，状态持久化
质量保障	多角度并行审查，分级处理（P1/P2/P3），持续验证（边做边测）
知识管理	即时文档化（趁上下文新鲜），分类存储（按问题类型），交叉引用（关联 Issue、PR）
工具设计	工具提供能力而非行为，Prompt 定义意图和流程，让代理决定如何达成目标

4.2 极简主义：设计理念如何影响我的实践

Claude Code 团队的实践给了我另一个启发：

> "最好的工具，就是没有工具。"

他们的做法：

- 只给模型一样东西：bash
- 每周都在删工具，因为新模型不需要了
- 减少模型的选择，就是增加模型的能力
- "模型吞噬脚手架"——曾经的外部辅助，逐渐被模型吸收

产品极简主义：不是"越来越丰富"，而是"越来越纯粹"。每一代模型发布，工具都会变得更简单，因为复杂性转移到了模型内部。

这个理念深刻影响了我做 AI 工程化的设计思路：

1. **入口极简化**：整个系统只有两个命令入口——`/req-dev` 和 `/optimize-flow`。不是因为功能少，而是把复杂性藏到了 Agent 的智能路由里。用户不需要记住十几个命令，只需要表达意图，Agent 会判断该调用哪个 Skill。
2. **Skill 而非工具堆叠**：speckit/openspec 倾向于提供更多工具、更多模板、更多约束。我选择相反的方向——把能力编码为 Skill，让 Agent 在需要时自动调用，而不是让用户手动选择"现在该用哪个工具"。
3. **上下文自动加载**：Claude Code 团队说"人类和 AI 看同样的输出，说同样的语言，共享同一个现实"。我把这个原则应用到上下文管理——不是让用户手动指定"加载哪些背景资料"，而是让 Agent 根据当前阶段自动加载相关的 context/。用户感受不到"上下文加载"这个动作，但 AI 已经具备了完整的信息。
4. **删除优先于添加**：每次迭代时，我会问自己"有哪些东西可以删掉？"而不是"还能加什么功能？"。AGENTS.md 从最初的长篇大论，精简到现在只放通用规范和目录指针，具体流程全部下沉到 Skill 里。

5. **双重用户设计**：Claude Code 为工程师和模型同时设计界面。AI 工程化也是——`/req-dev` 命令人可以手动调用，Agent 也可以在流程中自动调用子 Skill。同一套能力，两种调用方式，没有冗余。

当前实践的目标：让工具尽可能"隐形"——用户只需要说"我要做一个商品发放需求"，系统自动加载上下文、自动识别阶段、自动调用对应 Skill、自动沉淀经验。用户感受不到在"使用工具"，只是在"完成工作"。

> 注：关于工具消失的行业发展趋势，详见第九节"未来展望"。

4.3 上下文工程：AI 能力的前提是信息完整性

> 参考：**Anthropic - Effective Context Engineering for AI Agents**

什么是上下文工程？

上下文（Context） 指的是在从大语言模型（LLM）采样时包含的一组 token——不仅仅是提示词，还包括系统提示、工具定义、对话历史、检索到的文档等所有进入模型的信息。

上下文工程 是指在 LLM 推理过程中，策划和维护最优 token 集合的策略集合。它代表了 LLM 应用构建方式的根本转变：

提示词工程（旧范式）	上下文工程（新范式）
关注如何编写有效的提示词	管理整个上下文状态
主要针对一次性分类或文本生成任务	针对多轮推理和长时间运行的智能体
"找到正确的词语和短语"	"什么样的上下文配置最可能产生期望行为？"

核心指导原则：

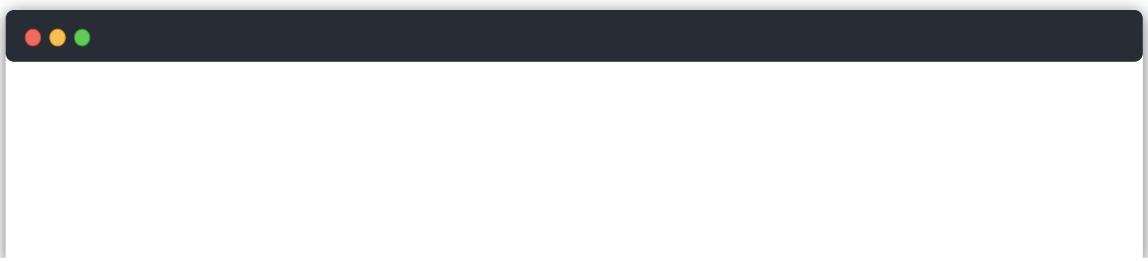
> **找到最小可能的高信号 token 集合，最大化期望结果的可能性**

为什么不重视上下文工程会导致严重问题？

很多团队把 AI 辅助编程的失败归咎于"模型不够强"或"提示词没写好"，但真正的根因往往是**上下文工程的缺失**。Anthropic 的研究揭示了几个关键问题：

问题 1：上下文腐蚀（Context Rot）

研究发现：**随着上下文窗口中 token 数量增加，模型准确回忆信息的能力会下降。**



```
上下文腐蚀的恶性循环：
加载更多信息 → 窗口膨胀 → 信息检索精度下降 → 行为异常
                    ↓
          人发现问题 → 加更多上下文纠正 → 窗口更膨胀 → 更差
```

这不是断崖式下降，而是梯度下降——模型在长上下文中仍然能力强大，但信息检索和长程推理的精度会持续降低。

问题 2：注意力预算耗尽（Attention Budget Exhaustion）

LLM 就像人类有限的工作记忆一样，拥有"注意力预算"：

```
Transformer 架构的约束：
├── 每个 token 都要关注所有其他 token，产生 n² 个成对关系
├── 训练数据中短序列比长序列更常见，模型对长上下文依赖的经验较少
└── 位置编码插值虽允许处理更长序列，但会降低 token 位置理解的精度

结果：
├── 每引入一个新 token 都会消耗注意力预算
├── 低质量的 token 会"稀释"高质量信息
└── 关键信息可能被噪声淹没
```

问题 3：speckit/openspec 的上下文盲区

回顾第二节的 speckit 困境，从上下文工程角度重新审视：

问题现象	上下文工程视角的根因
人 review 时逐步想起遗漏告诉 AI	历史经验没有编码为可检索的上下文
45 分钟完成需求，边际成本恒定	每次都是"冷启动"，没有上下文复用
上下文窗口频繁爆满	没有分层加载策略，一次性塞入过多信息
AI 行为异常，半途而废	上下文腐蚀导致关键信息被"遗忘"

问题 4：工具设计不当导致上下文污染

Anthropic 指出一个常见失败模式：

> "臃肿的工具集，覆盖过多功能或导致使用哪个工具的决策点模糊不清"

判断标准：**如果人类工程师无法明确说出在给定情况下应该使用哪个工具，AI 智能体也不能做得更好。**

```
工具设计不当的后果：
├── 工具描述冗长 → 消耗上下文预算
```

```
├── 工具边界模糊 → AI 决策困难，产生更多试错对话
├── 工具返回冗余信息 → 上下文快速膨胀
└── 最终：窗口爆满，任务失败
```

有效上下文工程的核心原则

基于 Anthropic 的实践和我们的落地经验，总结以下原则：

原则 1：分层式信息组织

```
context/
├── business/
|   └── 活动业务边界.md          ← 概要层（意图识别时加载）
├── tech/
|   └── Apollo配置规范.md        ← 技术层（方案设计时加载）
└── experience/
    ├── 商品发放历史问题.md      ← 经验层（实施前加载）
    └── 雅典娜配置注意事项.md    ← 详细层（配置时加载）
```

原则 2："即时"上下文策略（Just-in-Time Context）

不是预先加载所有可能相关的信息，而是维护轻量级索引，在运行时动态加载：

```
传统方式（预加载）：
启动 → 加载所有相关文档（20000 tokens）→ 开始工作 → 窗口已满一半

即时策略：
启动 → 加载索引文件（500 tokens）→ 识别当前阶段 → 按需加载（3000 tokens）
                                                        ↓
                                                窗口保持精简，信息高度相关
```

Claude Code 的实践：使用 `glob` 和 `grep` 等原语允许即时导航和检索文件，而不是预先加载完整数据对象到上下文中。

原则 3：上下文压缩与笔记系统

对于长时间运行的任务：

```
压缩（Compaction）：
├── 将接近上下文窗口限制的对话内容总结
├── 保留：架构决策、未解决的 bug、实现细节
├── 丢弃：冗余的工具输出或消息
└── 用摘要重新初始化新的上下文窗口

结构化笔记（Structured Note-taking）：
```

```
├── 智能体定期将笔记写入上下文窗口外的持久化存储
├── 稍后根据需要拉回上下文窗口
└── 实现跨压缩步骤的连贯性
```

原则 4：工具设计的上下文效率

```
好的工具设计：
├── 自包含：不依赖"记住"之前的对话
├── 返回精简：只返回 token 高效的必要信息
├── 边界清晰：用途明确，减少决策成本
└── 发挥模型优势：利用模型擅长的能力

坏的工具设计：
├── 返回完整数据库查询结果（可能数千行）
├── 工具描述长达数百 token
├── 多个工具功能重叠，边界模糊
└── 强迫模型做它不擅长的事情
```

上下文工程与 AI 工程化的关系

理解了上下文工程，就能理解 AI 工程化架构设计的"为什么"：

AI 工程化设计	上下文工程原理
context/ 分层目录	分层式信息组织，按阶段按需加载
Skill 封装固定流程	稳定执行过程，避免提示词遗漏导致的上下文不完整
Subagent 架构	主 Agent 保持精简，子任务独立窗口
状态文件传递	不依赖"记忆"，依赖结构化状态
经验沉淀机制	将知识编码为可检索上下文，而非依赖人脑

本质规律：

```
AI 的决策质量 ∝ 可用信息的完整性 × 信息的信噪比
                    ↑                    ↑
              不是越多越好          高信号、低噪声才有效
```

这意味着：

- 与其让人在 review 时逐步想起遗漏告诉 AI
- 不如建立系统化的上下文管理，让 AI 自动获取**精简且高信号**的信息

腾讯技术工程
腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。
688篇原创内容
公众号

五、实践：AI 工程化的设计与落地

5.1 AI 工程化是什么

经过反复思考和实践，我提炼出了 AI 工程化的定义：

> **智能化管理工作信息，以上下文工程的理解管理整个工作场景，借助AI的能力，降低人对已识别问题的处理成本**

组成部分：

1. 脚手架（Git 仓库形式）

- 把规范转为基础的目录结构
- 附带基础的初始化命令
- 存放业务线的上下文信息（业务背景、技术背景等）
- 随项目独立迭代的资源文件

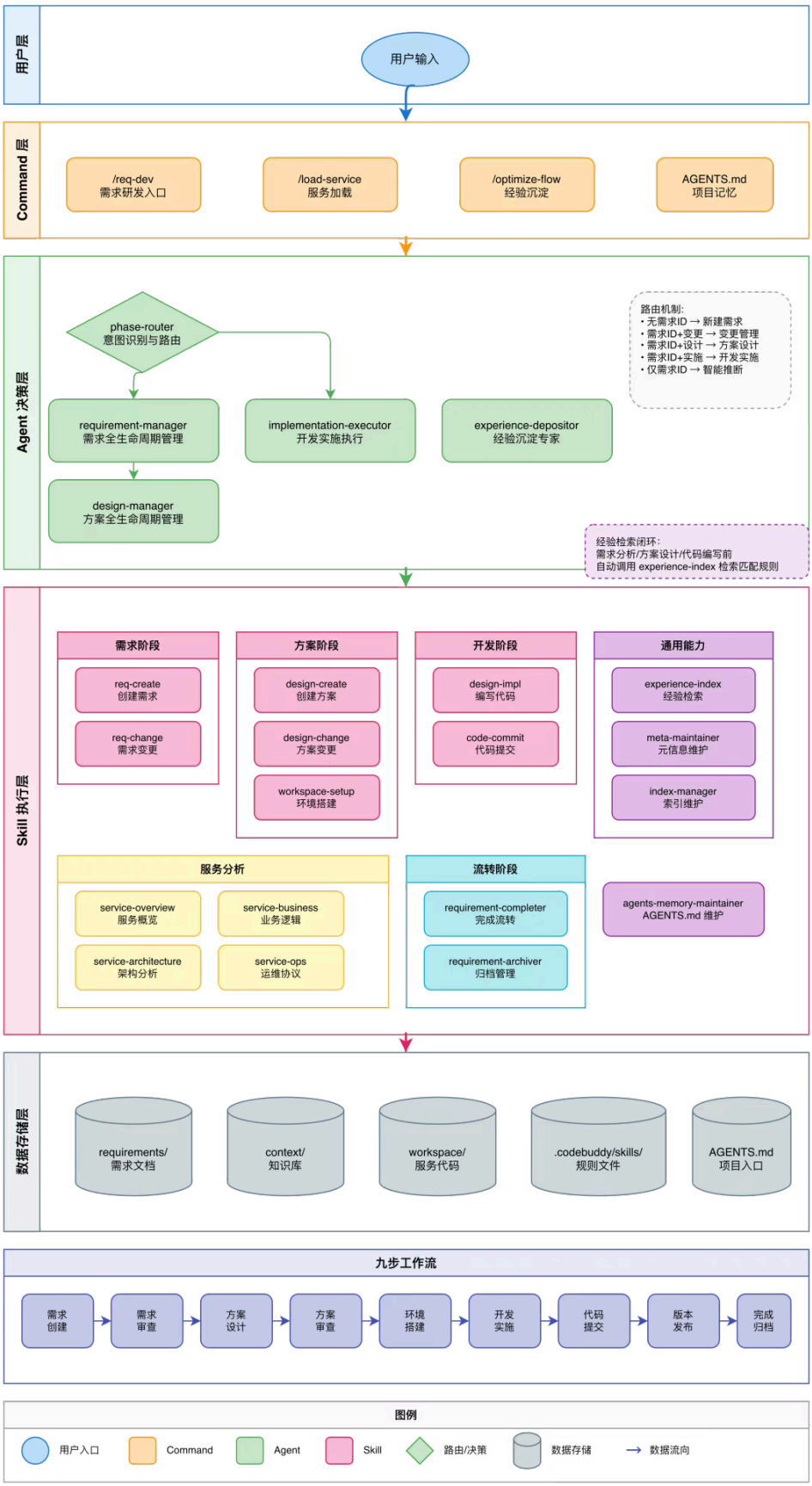
2. 工具包（插件形式）

- 提供 AI 工程需要的 cmd、skill、mcp、agent、hook 等
- 在插件市场迭代，分版本管理
- update 即可升级最新的规范、能力集成

为什么分脚手架和工具包？

- 插件市场内容会迭代、分版本，需要灵活升级
- 脚手架项目初始化后，随项目迭代，是独立的 git 仓库
- 脚手架适合存放基础资源文件和业务上下文信息
- 工具包适合封装通用能力和规范

AI 工程工具集系统架构



5.2 核心架构：Agent + Skill 分层设计



```
用户输入 → Command → Agent（决策层）→ Skill（执行层）
                          ↓
                    意图识别、流程路由
                          ↓
                    调用具体 Skill 执行
```

- **Agent**：自主决策层，负责意图识别、流程路由、上下文管理
- **Skill**：过程执行层，负责固定流程任务的具体执行
- **Command**：用户交互入口，通过 Agent 路由到具体执行

当前系统设计：

- **5 个 Agents**：phase-router、requirement-manager、design-manager、implementation-executor、experience-depositor
- **12 个 Skills**：req-create、req-change、experience-index、design-create、design-change、workspace-setup、design-implementation、code-commit、requirement-completer、requirement-archiver、meta-maintainer、index-manager
- **2 个 Commands**：`/req-dev`（需求研发统一入口）、`/optimize-flow`（流程优化沉淀）

5.3 目录结构：位置即语义

```
your-project/
├── AGENTS.md              # 项目记忆入口（每次会话自动加载）
├── .codebuddy/            # AI 自动化配置
|   ├── agents/            # Agent 定义（决策层）
|   ├── commands/          # 命令入口
|   └── skills/            # Skill 定义（执行层）
├── context/               # 项目知识库（长期记忆）
|   ├── business/          # 业务领域知识
|   ├── tech/              # 技术背景
|   |   └── services/      # 服务分析文档
|   └── experience/        # 历史经验
├── requirements/          # 需求管理
|   ├── INDEX.md           # 需求索引
|   ├── in-progress/       # 进行中需求
|   └── completed/         # 已完成需求
└── workspace/             # 代码工作区（Git 忽略）
```

三个核心约束：

1. **入口短小**：AGENTS.md 只放通用规范 + 目录指针，不写具体流程步骤
2. **位置即语义**：requirements/ 放需求产物，context/ 放可复用上下文，workspace/ 放代码
3. **复利沉淀**：每次执行命令，除了产出当前结果，还要让"下一次更快、更稳"

5.4 经验沉淀的技术实现

前面 4.1 节讲了复合工程的理念和三层沉淀机制，这里聚焦**具体怎么实现**。

触发时机：什么时候沉淀？

```
不是：做完需求后专门花时间"写总结"
而是：在流程关键节点自动触发沉淀

具体触发点：
├── 需求完成时 → requirement-completer skill 自动提取可复用经验
├── 遇到问题解决后 → 用户说"记住这个坑" → experience-depositor agent 记录
├── 代码提交时 → code-commit skill 检查是否有值得记录的模式
└── 流程优化时 → /optimize-flow 命令专门用于沉淀和优化
```

沉淀格式：记录什么？

```
# context/experience/商品发放-钱包选择问题.md

## 问题描述
商品发放时选错钱包类型，导致用户领取失败

## 触发条件
- 需求涉及商品发放
- 商品类型为虚拟商品

## 解决方案
虚拟商品必须发到虚拟钱包，实物商品发到实物钱包
具体判断逻辑见 Apollo 配置：xxx.wallet.type

## 校验方式
检查 goods_type 与 wallet_type 的匹配关系

## 关联文档
- context/tech/Apollo配置规范.md
- context/tech/services/商品服务技术总结.md
```

检索机制：怎么在对的时候加载？

检索由 `experience-index` Skill 统一负责，在需求分析、方案设计、代码编写前自动调用：

```
Agent 的上下文加载逻辑：

1. 意图识别阶段
   phase-router 识别意图，路由到对应 Agent
        ↓
2. 经验检索阶段
```

```
   Agent 调用 experience-index Skill，传入场景描述
   Skill 检索四类规则文件：
   ├── context-rules.md  → 匹配需加载的背景文档
   ├── risk-rules.md     → 匹配风险提示
   ├── service-rules.md  → 匹配服务依赖建议
   └── pattern-rules.md  → 匹配代码规范
         ↓
3. 返回结构化结果
   {
     "context": { "files": ["商品发放历史问题.md"] },
     "risk": { "alerts": [{"level": "high", "message": "注意钱包类型"}] },
     "service": { "suggestions": ["商品服务", "钱包服务"] },
     "pattern": { "files": ["error-handling.md"] }
   }
         ↓
4. Agent 主动提醒
   "注意：历史上商品发放有钱包选择问题，请确认..."
```

规则沉淀入口：通过 `/optimize-flow` 命令，调用 `experience-depositor` Agent 将新规则写入对应规则文件。

演进路径：从文档到 Skill 到 Command

```
阶段 1：纯文档（被动）
context/experience/xxx.md
→ AI 读取后提醒，但需要人确认

阶段 2：校验 Skill（半自动）
skill/product-distribution-validator
→ 自动校验配置，发现问题直接报错

阶段 3：完整 Command（全自动）
cmd/implement-product-distribution
→ 一个命令：加载背景 + 校验 + 生成 + 提醒 + 沉淀新经验

演进判断标准：
- 同类需求做了 5 次以上 → 考虑封装 Skill
- Skill 被调用 10 次以上 → 考虑封装 Command
- 不要过早抽象，让实践驱动演进
```

与 speckit 的本质区别

```
speckit 的知识流向：
人脑 → Spec 文档 → 代码
      ↑__________|
      下次还要从人脑开始
```

```
AI 工程化的知识流向：
人脑 → context/ → Skill → Command
          ↑_________|________|
          知识留在工具链里，下次直接复用
```

5.5 时间成本的量化对比

前面 2.5 节从"问题-方案"角度做了概念对比，这里从**时间成本**角度量化差异：

执行次数	speckit/open-spec	AI 工程化	累计节省
第 1 次	45 分钟	45 分钟（建立 context/）	0
第 2 次	45 分钟（人重新想）	15 分钟（部分复用）	30 分钟
第 5 次	45 分钟（还是要想）	5 分钟（大量复用）	130 分钟
第 10 次	45 分钟（...）	3 分钟（高度自动化）	315 分钟

关键差异：

- **知识位置**：speckit 在人脑（每次想），AI 工程化在 context/+skill/
- **新人上手**：speckit 依赖老人传授，AI 工程化第一天就能用
- **边际成本**：speckit 恒定，AI 工程化递减

六、深度对比：为什么传统 SDD 工具不够用

> 前面 2.5 节从"问题-方案"角度概述了 AI 工程化的优势，本节深入分析 speckit 和 openspec 的**技术设计缺陷**，帮助理解为什么需要新的解决方案。

6.1 speckit 的核心缺陷

问题 1：流程过于理想化

speckit 的 Constitution → Specify → Plan → Tasks → Implement 流程假设：

- 需求是清晰的
- 可以一次性规划
- 按阶段线性推进

但企业真实场景是：

- 需求动态变化

- 多方并行博弈
- 持续扯皮调整

问题 2：无法处理"考古"需求

speckit 从零开始定义，但真实开发必须"考古"：

- 历史坑点在哪？
- 现有能力有哪些？
- 配置规范是什么？

问题 3：知识不会沉淀

```
每次执行：Constitution → Specify → Plan → Tasks → Implement
                                                    ↓
                                                每次从头开始
```

缺失机制：

- ❌ 实施过程中发现的坑不会被记录
- ❌ 排查信息丢失
- ❌ 下次遇到类似问题还得重新排查

问题 4：宪章系统的僵化

9 条不可变原则固然保证质量，但：

- ✅ 适合标准化项目（Demo、开源库）
- ❌ 不适合企业定制场景（历史债务、框架限制、合规要求）

6.2 openspec 的核心缺陷

问题 1：Delta 机制的理论美好与现实骨感

假设需求可以"提案化"，但企业真实场景是多线并行、动态调整、持续扯皮。

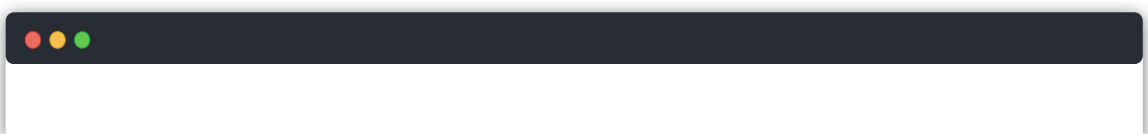
问题 2：Fail-Fast 的代价

理论上保证一致性，实际上成为阻塞点。人的认知窗口有限，很难手动解决复杂冲突。

问题 3：强依赖命名的脆弱性

产品、运营、研发对同一个需求有不同表述，命名不一致导致归档失败。

问题 4：Archive 只是"合并"，不是"学习"



```
F(CurrentSpec, DeltaSpec) → NewSpec

缺失的维度：
F(CurrentSpec, DeltaSpec, Context, Lessons) → NewSpec + Knowledge
                                ↑           ↑
                          实施上下文     经验教训
```

6.3 共性问题：忽视人的现实工作模式

问题 1：忽视认知负担管理

两个工具都假设人能理解并遵循复杂流程、维护大量结构化文档、记住所有规范和约束。

但现实是：土办法最管用。工具应该适配人的工作模式，而不是强行改变它。

问题 2：忽视"执行过程"的价值

只关注"规范"和"结果"，忽视"过程"中的知识价值。

问题 3：忽视复利效应的关键性

```
传统工具：帮你"做事"
复合工程：帮你"越做越快"

传统工具：每次都是新的开始
AI 工程化：每次都站在上次的肩膀上
```

问题 4：Spec 详细程度的悖论

规范驱动开发有一个根本性的矛盾：

```
Spec 越详细 → 越接近代码本身 → 维护两份"代码"
Spec 越简略 → 越难指导 AI → 失去规范的意义
```

详细 Spec 的问题：

- 当 Spec 详细到可以精确指导 AI 生成代码时，它本身就变成了另一种形式的"代码"
- 你需要同时维护 Spec 和 Code 两套产物，且要保持同步
- 代码改了 Spec 要改，Spec 改了代码要改——双倍维护成本

AI 工程化的解法：不追求详细 Spec，而是**分层概要 + 代码指针**

```
AI 工程化的上下文组织：
├── 服务概要：这个服务做什么、边界在哪
```

```
├── 业务概要：核心业务流程、关键概念
├── 模块概要：模块职责、依赖关系
├── 接口概要：对外接口、调用方式
└── 代码指针：具体细节在 xxx/xxx.go 的 xxx 函数

不维护：
├── ❌ 详细的数据结构定义（代码里有）
├── ❌ 完整的接口参数说明（代码里有）
├── ❌ 具体的实现逻辑描述（代码里有）
└── ❌ 任何可以从代码直接获取的信息
```

核心原则：概要层帮助 AI 快速定位，细节层直接读代码。避免维护一份"像代码一样详细的 Spec 文档"——那只是换了个格式的代码，没有降低复杂度，反而增加了同步成本。

七、进阶能力：插件、Skill、MCP 的融合

对于大多数研发同学来说，可能还停留在 speckit、openspec 这类规范驱动工具的认知上。但 AI 工程化把更多能力融合在了一起：

7.1 Skill：可复用的能力单元

Skill 是过程执行层的基本单元，每个 Skill 负责一个具体的固定流程任务：

```
.codebuddy/skills/
├── req-create/              # 需求创建
|   ├── SKILL.md            # 技能定义
|   └── templates/          # 模板资源
├── design-create/           # 方案创建
├── workspace-setup/         # 环境搭建
└── code-commit/             # 代码提交
```

Skill 的特点：

- **单一职责**：每个 Skill 只做一件事
- **可复用**：多个流程可以调用同一个 Skill
- **可组合**：复杂流程由多个 Skill 组合完成
- **可演进**：Skill 可以独立升级，不影响其他部分

7.2 Agent：自主决策层

Agent 负责意图识别、流程路由、上下文管理：

```
```

```
.codebuddy/agents/
├── phase-router.md         # 阶段路由，意图识别
├── requirement-manager.md # 需求全生命周期管理
├── design-manager.md       # 方案全生命周期管理
├── implementation-executor.md # 开发实施执行
└── experience-depositor.md     # 经验沉淀（独立上下文）
```

Agent 与 Skill 的分工：

- **Agent**：决定"做什么"
- **Skill**：执行"怎么做"

7.3 多 Agent 协作：从上下文窗口爆满到高效分工

在实践 AI 工程化的过程中，我们遇到了一个关键瓶颈：**上下文窗口爆满**。

问题的根源

早期使用 speckit 等工具时，最痛苦的体验是：

```
执行复杂需求时：
├── 加载业务背景（5000 tokens）
├── 加载技术上下文（8000 tokens）
├── 加载历史经验（3000 tokens）
├── 当前对话记录（持续增长）
└── ...
        ↓
窗口频繁爆满 → 强制截断 → 丢失关键上下文 → AI 行为异常
```

Anthropic 工程团队精准描述了这个问题：

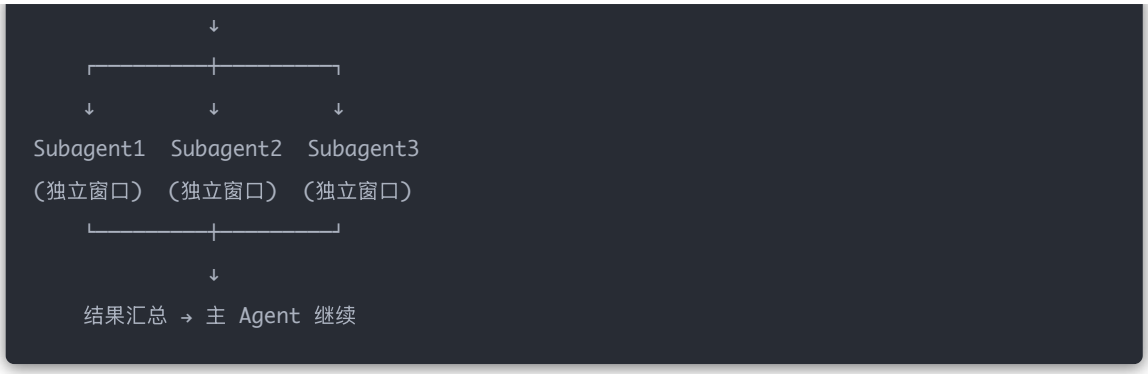
> "想象一个软件项目由轮班工程师负责，每个新工程师到来时对上一班发生的事情毫无记忆。"

解决方案：Subagent 架构

借鉴 Anthropic 的双 Agent 架构思想，我们设计了 **主 Agent + Subagent** 的协作模式：

```
传统模式（单一 Agent）：
用户输入 → 一个大 Agent 处理所有事情 → 上下文持续膨胀 → 窗口爆满 → 任务失败

Subagent 模式：
用户输入 → 主 Agent（决策层）
                ↓
    意图识别 + 任务拆分
```



核心优势:

特性	说明
独立上下文窗口	每个 Subagent 有自己的上下文空间，不会互相污染
专注单一任务	每个 Subagent 只处理一件事，认知负担小
并行执行	多个 Subagent 可以同时工作，提升效率
结构化状态传递	通过文件传递结果，而非依赖"记忆"

效果对比

指标	单 Agent 模式	Subagent 模式
窗口爆满频率	70%（复杂需求几乎必爆）	5%（偶发于极端场景）
任务完成率	60%（经常中途失败）	95%（可靠完成）
上下文利用效率	30%（大量冗余信息）	80%（按需加载）

状态传递机制

Subagent 之间不共享上下文窗口，通过**结构化状态文件**保证信息传递:

```
核心文件:
├── requirements/INDEX.md       # 需求状态索引
├── requirements/in-progress/   # 进行中的需求详情
└── context/session/            # 会话级临时上下文

工作流程:
1. Subagent 启动时：读取状态文件，快速理解当前状态
2. Subagent 执行中：专注自己的任务
3. Subagent 结束时：更新状态文件，提交"干净的交接"
```

核心原则：每个 Subagent 只完成一个"原子任务"，不是一个工程师连续工作 48 小时，而是轮班工程师每人 4 小时但交接清晰。

与 speckit 的本质差异

```
speckit：依赖"一个 Agent 记住所有事情"
         Constitution → Specify → Plan → Tasks → Implement
         上下文持续累积，到 Implement 阶段时窗口已经很满

Subagent：依赖"结构化的状态传递"
         每个阶段独立的 Subagent，独立的上下文窗口
         状态通过文件传递，而非上下文累积
```

前者是人脑模型（记忆有限），后者是团队协作模型（交接清晰）。

7.4 MCP：外部系统集成

MCP（Model Context Protocol）让 AI 能够直接对接外部系统：

```
基础集成：
├── TAPD MCP（需求管理）
|   ├── 自动获取需求详情
|   ├── 关联相关需求
|   └── 更新需求状态
├── 工蜂 MCP（代码管理）
|   ├── 自动创建分支
|   ├── 提交代码变更
|   └── 创建合并请求
└── iWiki MCP（知识管理）
    ├── 检索历史技术方案
    ├── 获取业务背景文档
    └── 关联团队知识库
```

MCP 的价值：

- **自动化操作**：不需要人手动操作 TAPD、工蜂、iWiki
- **信息同步**：AI 自动获取最新信息
- **减少错误**：避免手动操作的遗漏和错误

7.5 插件市场：能力的分发与升级

工具包以插件形式发布到插件市场：

- **版本管理**：每个版本独立，可回滚
- **灵活升级**：update 即可获得最新能力
- **团队共享**：团队成员共享同一套能力集

与脚手架的配合：

- 脚手架存放业务上下文（随项目迭代）
- 工具包提供通用能力（独立版本管理）



腾讯技术工程

腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。

688篇原创内容

公众号

八、落地策略：从零到一的实践路径

> 前面各节从理论角度阐述了 AI 工程化的设计，本节聚焦**具体怎么落地**。以 2.5 节提到的"商品发放"场景为例，展示完整的实践路径。

8.1 冷启动：新项目接入

> 冷启动是 AI 工程化的核心优势之一。传统工具的知识在人脑，需要传授；AI 工程化的知识在工具链里，开箱即用。

步骤 1：安装 AgentProjectKit 插件（5 分钟）

首先需要添加插件市场并安装 AgentProjectKit：

```
# 安装 AgentProjectKit 插件
/plugin install agent-project-kit@tmap-codebuddy-plugin

# 验证安装
/plugin list
```

步骤 2：脚手架初始化（15 分钟）

```
# 初始化 AI 工程项目
/agent-project-kit:init-project
```

命令会自动完成：

1. 克隆 AI 工程项目模板
2. 引导配置项目基本信息（业务线名称、定位等）
3. 初始化 AGENTS.md 项目记忆文件

步骤 3：加载服务上下文（30 分钟）

这是冷启动的关键步骤。`/agent-project-kit:load-service` 命令实现**项目级别长期记忆初始化**：

```
# 加载相关服务，生成技术总结
/agent-project-kit:load-service
/agent-project-kit:load-service
/agent-project-kit:load-service
```

`/agent-project-kit:load-service` 的工作流程：

```
用户执行 /agent-project-kit:load-service
                ↓
1. 克隆服务代码到 workspace/loadservice/ 目录
                ↓
2. 分析服务架构、业务逻辑、API 接口：
   - 业务定位、核心职责、技术栈
   - 依赖关系、对外接口、数据模型
   - 关键模块、配置要点、常见坑点
                ↓
3. 生成技术文档到 context/tech/services/ 目录
                ↓
结果：AI 获得该服务的完整上下文，后续任何涉及该服务的需求
      都会自动加载这份上下文
```

为什么这很重要？

- speckit/openspec：每次需要描述服务背景时，依赖人记住并手动描述
- AI 工程化：一次 `/agent-project-kit:load-service`，永久复用，新成员也能立即获得"老兵视角"

步骤 4：开始需求研发

使用 `/req-dev` 命令开始你的第一个需求：

```
# 创建新需求
/req-dev 实现用户认证功能

# 或者指定已有需求继续工作
/req-dev REQ-001
```

工具包自带**常用研发工具集成（MCP）**，开箱即用：

MCP 集成	功能	传统方式
TAPD MCP	自动获取需求详情、关联需求、更新状态	手动复制粘贴需求内容
工蜂 MCP	自动创建分支、提交代码、创建 MR	手动操作 Git 命令
iWiki MCP	检索历史技术方案、业务背景文档、团队知识库	手动搜索翻阅 Wiki 页面

MCP 集成的价值:

- 不是"又多了几个工具要学"，而是"AI 自动帮你操作这些系统"
- 需求来了 → AI 自动从 TAPD 拉取详情 → 自动检索 iWiki 历史方案 → 自动生成方案
- 人只需要 review 和确认

冷启动效果对比:

阶段	speckit/openspec	AI 工程化
学习工具	1-2 小时	5 分钟（插件安装）
初始化项目	手动搭建	15 分钟（/agent-project-kit:init-project）
了解服务架构	2-4 小时（需老人讲解）	30 分钟（/agent-project-kit:load-service 自动分析）
准备总计	4-7 小时	50 分钟
首次工作质量	不稳定（依赖记忆和传授）	稳定（context/ 提供完整信息）

关键差异:

- speckit/openspec：工具是"空壳"，知识在人脑，需要传授
- AI 工程化：工具包含"知识"（context/+MCP），**新人第一天就能高质量工作**

8.2 持续迭代：知识的复利沉淀

第 1 个需求：建立 context/

```
需求：实现 12 月活动的商品发放

执行过程中发现问题：
- Apollo 配置有特殊格式要求
- 雅典娜 20 种商品类型，配置方式各不同
- 钱包选择要区分虚拟/实物
- 敏感 接口有合规要求
```

```
知识沉淀：
人："@agent，记住这些坑"
    ↓
自动生成/更新 context/：
├── context/tech/Apollo配置规范.md
├── context/experience/雅典娜配置注意事项.md
├── context/experience/商品发放历史问题.md
└── context/business/跨团队协作.md

耗时：45 分钟（首次建立）
```

第 2 个需求：复用 context/

```
需求：实现春节活动的商品发放（类似场景）

AI 自动加载 context/，自动提醒历史坑点
人 review："嗯，都考虑到了" ✓

新发现：春节活动需要限制地域
    ↓
"@agent，记住地域限制"
    ↓
context/ 自动更新

耗时：15 分钟（大量复用，少量新增）
```

第 6-10 个需求：封装为 skill

```
当 context/ 足够完善，封装为能力层：

skill/product-distribution-helper:
- 自动加载所有商品发放相关 context/
- 自动校验 Apollo 配置格式
- 自动检查雅典娜商品类型
- 自动提醒钱包选择、地域限制
- 自动生成监控配置

使用：/implement-product-distribution → 一键完成

耗时：3 分钟（高度自动化）
```

8.3 团队协作：知识的共享与传承

新成员第一天：

```
```

```
speckit/openspec:
1. 学习工具用法（1-2 小时）
2. 了解服务架构（需老人讲解，2-4 小时）
3. 熟悉流程规范（1 小时）
4. 开始工作：依赖记忆和老人传授，首次质量不稳定
总计：4-7 小时准备 + 不稳定的首次质量

AI 工程化：
1. 脚手架初始化（15 分钟）
2. 工具包安装（5 分钟）
3. 立即开始工作：
   - context/ 提供服务上下文
   - MCP 自动集成 TAPD/工蜂/Apollo
   - cmd/skill 引导完成任务
   - 首次就能高质量完成
总计：20 分钟准备 + 稳定的首次质量
```

团队效应：

```
5 人团队，各做 2 次商品发放：

speckit：5 人 × 2 次 × 45 分钟 = 450 分钟

AI 工程化：
第 1 人第 1 次：45 分钟 → context/ 建立
第 1 人第 2 次：15 分钟
第 2 人第 1 次：15 分钟（复用第 1 人 context/）
第 2 人第 2 次：10 分钟
...
第 5 人第 2 次：5 分钟

总计：126 分钟
节省：450 - 126 = 324 分钟（72%）
```

九、未来展望：工具终将消失

> 第 4.2 节讨论了极简主义如何影响当前设计，本节从**行业发展趋势**角度展望工具的演进方向。

9.1 模型吞噬脚手架

随着模型能力的提升，很多外部辅助会被模型内化：

```
```

```
Opus 4.1 需要的东西，Sonnet 4.5 已经内化了
    ↓
系统提示可以删 2000 个 tokens
    ↓
工具每周都在变简单
```

这意味着什么？ 今天我们在 context/、Skill、Agent 中编码的知识和流程，未来可能直接被模型"学会"。AI 工程化的架构设计需要为这种迁移做好准备——当某个 Skill 不再需要时，能够平滑删除而不影响整体。

9.2 多 Agent 架构的演进方向

从"工具调用"到"团队协作"

当前的 AI 辅助编程主要是"人调用 AI"模式：

```
人 → 发指令 → AI 执行 → 人检查 → 人发下一个指令
```

Subagent 架构开启了新的可能：

```
人 → 设定目标 → 主 Agent 拆解 → 多个 Subagent 协作 → 主 Agent 汇总 → 人验收
```

未来可能演进为：

```
人 → 设定目标 → Agent 团队自主协作数小时/数天 → 人验收最终结果
```

长时间运行 Agent 的关键挑战

Anthropic 的实践揭示了几个核心挑战：

挑战	当前解法	未来方向
上下文窗口限制	Subagent 分解 + 状态文件传递	更高效的 compaction + 更智能的上下文选择
任务连续性	结构化状态文件（JSON/Markdown）	更丰富的"工作记忆"机制
质量保证	端到端测试 + 人工 Review	专门的 QA Agent + 自动化验收
错误恢复	状态文件支持断点续做	更智能的错误分析和自动修复

Agent 专门化 vs 通用化的权衡

一个开放问题：应该用一个强大的通用 Agent，还是多个专门化的 Agent？

```
通用 Agent 路线：
├── 优势：简单，不需要协调
├── 劣势：上下文负担重，需要"知道所有事情"
└── 适合：简单任务、短时间任务

专门化 Agent 路线：
├── 优势：每个 Agent 更专精，上下文更精简
├── 劣势：需要协调机制，状态传递成本
└── 适合：复杂任务、长时间任务、团队协作场景
```

我们的选择：对于企业级复杂场景，专门化 Agent 更适合。原因是：

1. 企业场景本身就是"团队协作"，Agent 架构应该反映这一现实
2. 上下文窗口是硬约束，专门化可以更高效利用
3. 专门化 Agent 更容易独立迭代和优化

与人类团队的类比

最好的 Agent 架构设计，灵感来自人类高效团队的工作方式：

```
人类团队：
├── 产品经理：理解需求、拆解任务
├── 技术 Leader：设计方案、分配工作
├── 开发工程师：实现功能
├── 测试工程师：验证质量
└── 每个人有自己的专业领域，通过"会议"和"文档"协调

Agent 团队：
├── phase-router：理解意图、路由任务
├── design-manager：设计方案
├── implementation-executor：实现功能
├── test-agent（计划中）：验证质量
└── 每个 Agent 有自己的专业 Prompt，通过"状态文件"协调
```

> Anthropic 工程团队的洞察："这些实践的灵感来自于了解高效软件工程师每天做什么。"

当前范式：Claude 做一步，你检查，批准，它继续。

未来范式：

```
```

```
当模型可以自主工作几天甚至几周：
早上："我想完成 X"
晚上：看结果

中间的过程？它自己处理。
```

人的角色从"操作者"变成"监督者"，从"指令发出者"变成"目标设定者"。

AI 工程化的定位：在这个转型过程中，AI 工程化是"过渡期基础设施"——帮助团队在当前阶段高效工作，同时为未来的全自动化积累知识和经验。

9.3 研发工作的本质变化

AI 工程化不只是引入新工具，而是重新定义了研发的工作方式。这种变化已经在 AI 技术最前沿的团队中发生。

首先要避免的认知误区

工程师在使用 AI 时最常见的两种误解：

误区	表现	结果
AI 是"银弹"	期望 AI 自动理解需求、写出完美代码	过度依赖，缺乏监督，质量不稳定
AI 是"思考替代品"	把 AI 当作可以替代人类思考的工具	不理解业务，一直捣鼓 AI，适得其反

正确的定位是：**AI 是强大的执行工具，但决策权和判断力必须留在人手中。**

来自 OpenAI 与 Anthropic 的实践经验

理解 AI 的真实能力边界

参考 OpenAI 团队使用 Codex 构建 Sora 安卓应用的经验，将 AI 定位为**"一位新入职的资深工程师"**：

需要人类指导	表现卓越
无法推断隐性上下文（团队偏好、内部规范）	快速理解大型代码库，精通主流编程语言
缺乏真实用户体感（无法感知"滚动不顺畅"）	热衷于编写单元测试，能根据 CI 日志修复问题
深层架构判断力不足（本能是"让功能跑起来"）	支持大规模并行，同时探索多种方案

三步协作工作流（借鉴 OpenAI 与 Anthropic 经验）：

阶段	人的职责	AI 的职责
奠定基石	定义架构、编写范例代码、设定标准	学习并遵循
共同规划	校准理解、确认方案	总结现状、生成设计文档
执行交付	架构把关、质量审查	编码实现、测试修复

Anthropic 内部调查数据（2025年8月，132名工程师，20万条使用记录）：

- 工程师在 **60% 的工作中使用 AI**，实现 **50% 的生产力提升**，年同比增长 2-3 倍
- **27% 的 AI 辅助工作是原本不会完成的任务**（如交互式仪表板、探索性工作）
- 工程师倾向于委托**易于验证、定义明确、代码质量不关键、重复无聊**的任务

> "我可以非常胜任前端、事务性数据库的工作...而以前我会害怕触碰这些东西。" —— 后端工程师"我以为我真的很享受编写代码，但实际上我只是享受编写代码带来的结果。" —— 高级工程师

核心理念：寻找 AI 的"舒适区"

工程师的核心工作之一，已经从纯粹的编码转变为**识别 AI 的能力边界，并将复杂任务转化为落入 AI "舒适区"内的子任务**：

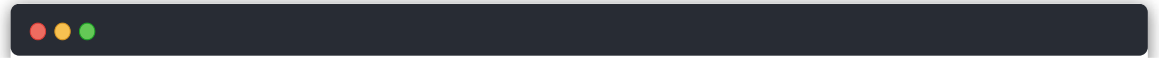
- **低标准、高容错场景**：任务对精确度要求不高，容忍多次失败。AI 尝试 N 次只要一次成功，就是显著提效
- **迭代式开发场景**：形成"AI 初步实现 → 人验证修正 → 快速反馈"的闭环，不追求一次完美

工作模式的具体变化

工作内容的迁移：

工作环节	传统模式	AI 工程化模式	角色变化
需求理解	反复阅读文档、追问产品	Agent 自动加载 context/，主动提示	信息收集者 → 信息确认者
方案设计	从零构思、翻阅历史代码	基于模板生成，AI 提示已知风险	方案起草者 → 方案审核者
代码实现	逐行编写、查文档、调试	AI 生成初版，人 review 调整	代码生产者 → 代码把关者
知识沉淀	写文档（经常忘记）	`/optimize-flow` 即时沉淀	文档维护者 → 经验触发者

时间分配的重构：



```
传统研发：                          AI 工程化后：
├── 40% 信息收集                    ├── 10% 信息确认
├── 30% 重复劳动                    ├── 10% 结果审核
├── 20% 核心决策          →         ├── 50% 核心决策
└── 10% 知识沉淀                    └── 30% 知识沉淀
```

一个具体的对比——以"商品发放需求"为例：

```
传统模式的一天：                    AI 工程化模式的一天：
09:00-10:30 阅读需求文档，追问产品   09:00-09:30 /req-dev，确认需求边界
10:30-12:00 翻阅历史代码，理解逻辑   09:30-10:30 review AI 方案，调整决策点
14:00-15:30 询问老人"以前怎么做"     10:30-12:00 review AI 代码，优化核心逻辑
15:30-18:00 写代码，边写边查文档     14:00-15:00 AI 辅助测试，修复问题
18:00-19:00 遇到配置问题，排查       15:00-15:30 /optimize-flow 沉淀经验
19:00-20:00 继续写代码               15:30-17:00 处理下一个需求
产出：完成 60%，知识留在脑子里       产出：完成 100%，经验沉淀到 context/
```

能力要求的升级

能力维度	传统要求	AI 工程化要求
编码能力	熟练编写各类代码	能判断 AI 生成代码的质量和风险
知识储备	记住各种细节和坑点	知道如何组织知识让 AI 能用
问题解决	自己动手排查	会描述问题让 AI 辅助分析
效率提升	写更多代码、加更多班	设计更好的 Skill、沉淀更多经验

新的核心竞争力体现为三种能力：

1. **系统理解能力**：AI 能实现功能，但只有人能判断它是否以正确方式融入系统
2. **AI 协作能力**：设计上下文、拆解计划、通过反馈循环持续优化
3. **设计质量标准**：当"写出能工作的代码"门槛降低，架构设计和交付质量成为区分标准

监督悖论：有效使用 AI 需要监督能力，而监督能力可能因过度依赖 AI 而退化。Anthropic 的一些工程师故意在没有 AI 的情况下练习以"保持敏锐"。

本质洞察

黄仁勋有一个精准的判断：**AI 改变的是"任务"，而非"职业"**。

- **被 AI 接管的任务**：信息检索、样板代码、格式化、重复配置
- **人依然主导的核心**：系统设计、架构决策、质量判断、创新突破

AI 工程化的价值，就是让这种"任务迁移"在团队中系统化落地——通过 context/ 让信息检索自动化，通过 Skill 让重复流程标准化，通过经验沉淀让知识持续复利。

最终目标：让研发把时间花在"只有人能做的事"上，而不是"AI 也能做的事"上。

9.4 工具隐形化：从"使用工具"到"完成工作"

工具消失的含义：不是工具不存在了，而是工具变得如此无缝，你感受不到它的存在。

```
就像现在你用搜索引擎，不会想"我在使用一个信息检索系统"。
你只是在找答案。工具隐形了。
```

隐形化的三个层次

层次一：操作隐形——从"记住命令"到"表达意图"

```
过去：记住 20 个命令，选择正确的那个
├── /speckit.constitution
├── /speckit.specify
├── /speckit.plan
├── /speckit.tasks
└── ...

现在：只说你要什么
├── "/req-dev 实现商品发放" → Agent 自动判断是创建还是继续
└── 不需要知道底层调用了哪些 Skill
```

层次二：知识隐形——从"想起经验"到"系统提醒"

```
过去：做需求时，人要想起历史上有什么坑
├── "上次商品发放好像有个钱包问题..."
├── "Apollo 配置格式是什么来着..."
└── 认知负担在人身上

现在：experience-index 自动检索，主动提醒
├── "检测到商品发放场景，已加载相关经验..."
├── "风险提示：注意钱包类型匹配"
└── 知识在系统里，人只需确认
```

层次三：流程隐形——从"遵循步骤"到"自然完成"

```
过去：严格按 Constitution → Specify → Plan → Tasks → Implement 执行
├── 人要知道"现在该执行哪个阶段"
```

```
├── 人要判断"前置条件是否满足"
└── 流程感知在人身上

现在：Agent 自主决策流程路由
├── 人说"继续做 REQ-001"
├── phase-router 自动判断当前阶段和下一步
└── 人感受到的是"工作在推进"，而非"在执行流程"
```

AI 工程化的隐形化进度

维度	当前状态	目标状态
命令入口	✅ 2 个命令覆盖全流程	自然语言直接触发
上下文加载	✅ experience-index 自动检索	完全无感知加载
阶段流转	✅ phase-router 自动路由	Agent 自主推进多步
经验沉淀	🔄 需要 /optimize-flow 触发	自动识别并沉淀
跨会话连续性	🔄 依赖状态文件	无缝断点续做

隐形化的终极形态

```
今天：
人："我要做一个商品发放需求"
AI：执行一步，等待确认
人：确认，继续
AI：执行下一步，等待确认
...

明天：
人："我要做一个商品发放需求"
AI：分析、设计、实现、测试、提交 PR
人：Review 最终结果

后天：
人：（在 TAPD 创建需求单）
AI：（自动感知、自动完成、自动提交 Review）
人：（只在关键决策点介入）
```

最后一步：你不再"使用"工具，你只是在思考业务问题，而工具已经把代码写好了。

十、写在最后：从第一性原理出发

回顾这段历程，我最大的收获是：**不要为了用工具而用工具**。

speckit 和 openspec 都是优秀的工具，它们定义的流程、模板、检查清单都很有价值。但正如 2.5 节的对比所示，它们解决的是"规范化"问题，而企业真实场景的核心问题是：

1. **上下文缺失**：AI 看不到历史经验、业务边界、配置规范
2. **知识不沉淀**：每次都从头开始，边际成本恒定
3. **范围太窄**：只管单个仓库，无法覆盖跨服务、跨系统的复杂场景

AI 工程化试图解决这些问题：

```
上下文工程 → 让 AI 自动获取完整信息
复合工程 → 让每次实践都降低下次成本
项目级方案 → 管理所有仓库和外部系统
```

核心思路：

```
能够落地的最高效流程 → 已存在于高效的人的行为过程中
                    ↓
        把高效流程 AI 化 → 推广到全团队应用
                    ↓
   细节流程在具体业务线中迭代 → 自定义探索
                    ↓
   实践中发现问题 → 提取可复用信息 → AI 工程化融入工具
                                        ↓
                          下次通用场景使用时可复用
```

最后想说的是：

AI 工程化不是要替代 speckit 或 openspec，而是在它们的基础上，融合上下文工程、复合工程、插件市场、MCP 集成等能力，形成一套更适合企业复杂场景的解决方案。

如果你也在探索 AI 辅助研发，希望这篇文章能给你一些启发：

1. **从真实工作场景出发**，而不是从工具出发
2. **把知识编码进工具**，而不是只写文档
3. **追求边际成本递减**，而不是固定成本
4. **让工具适配人**，而不是让人适配工具

工具的终极形态是消失。在那一天到来之前，我们要做的是让工具越来越"懂"我们的工作，越来越"记得"我们的经验，越来越"自然"地融入我们的日常。

这就是 AI 工程化的意义所在。

参考资料

- 上下文工程
- 复合工程
- **claude code团队复合工程**
- **openspec**
- **speckit**
- 多Agent协作的长时间任务
- 我们如何使用 Codex 在 28 天内构建 Android 版 Sora
- **SKILL**



腾讯技术工程
腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。
688篇原创内容
公众号



阅读 1.6万

留言

写留言

还没有留言，来写首评